# Section Overview

- Applying the Roofline Model: Block Tiling and Thread Coarsening

# Driving Question

## How can we incrase AI?

The roofline model tells us that increasing AI should increase throughput.

**But how can we actually increase AI?**

# Driving Question

## How can we incrase AI?

The roofline model tells us that increasing AI should increase throughput.
**But how can we actually increase AI?**

1. Utilizing SMEM more aggressively.

# Driving Question

## How can we incrase AI?

The roofline model tells us that increasing AI should increase throughput.
**But how can we actually increase AI?**

1. Utilizing SMEM more aggressively.
2. Loading larger blocks into SMEM.

# Driving Question

## How can we incrase AI?

The roofline model tells us that increasing AI should increase throughput.
**But how can we actually increase AI?**

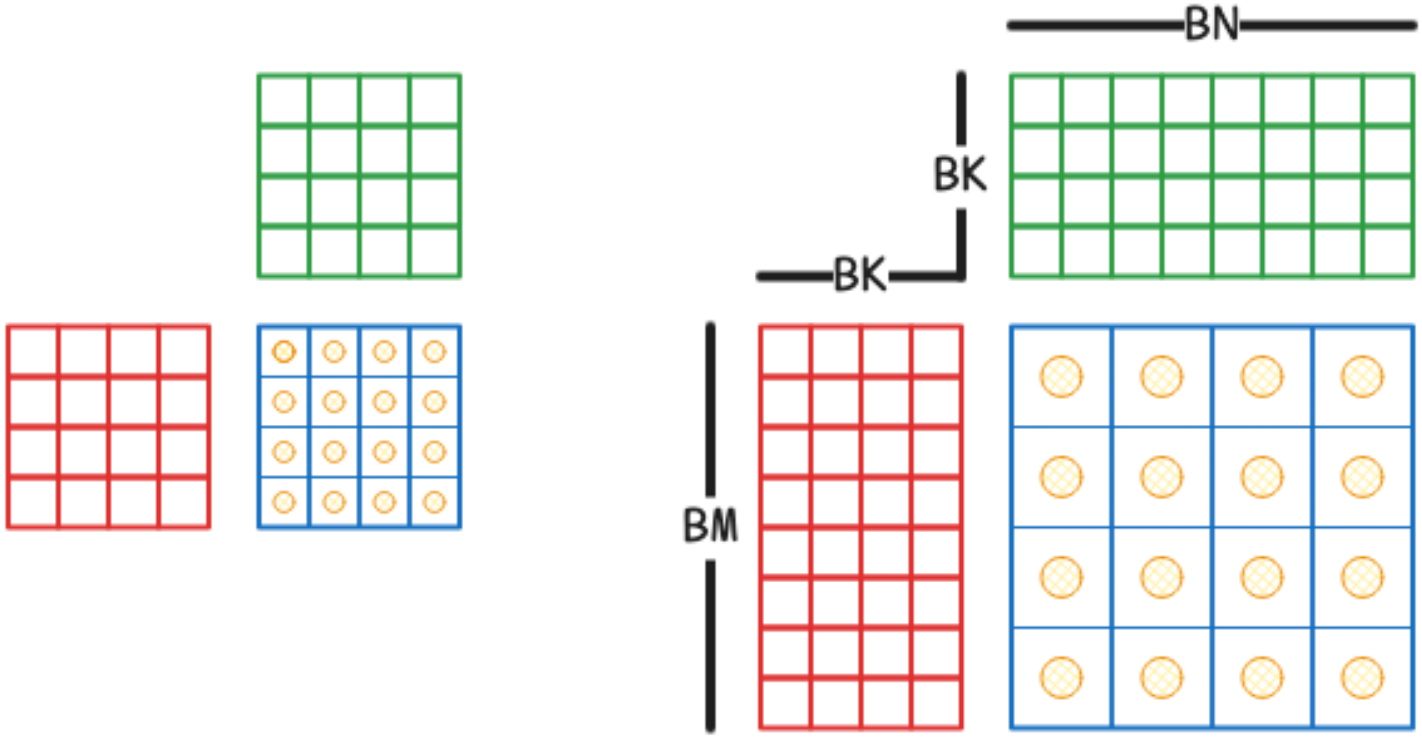
1. Utilizing SMEM more aggressively.
2. Loading larger blocks into SMEM.
3. Reusing values in SMEM to compute multiple values of $C$.

# Generalized Tiling: Loading larger blocks.

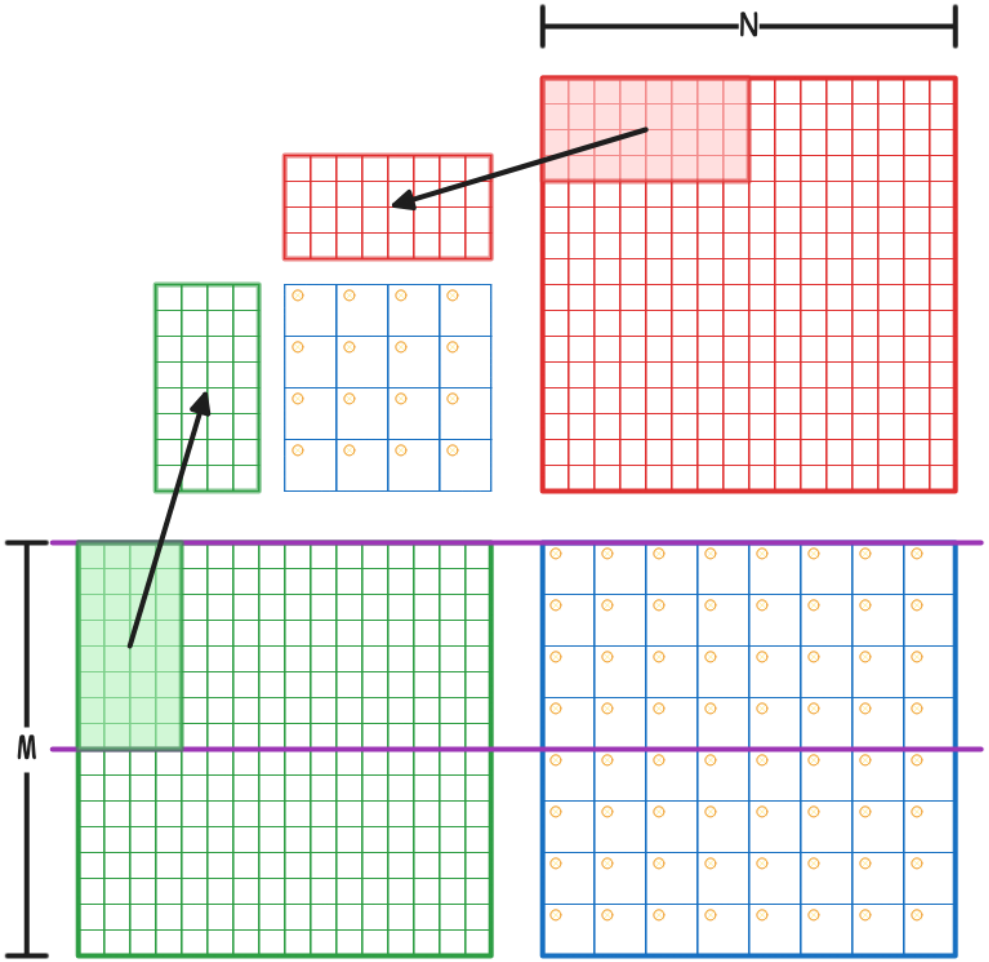
Our tiled kernel **hardcoded** a square $32 \times 32$ block size. Each thread in the block load one value into SMEM. We can parameterize the tiling with $BM$, $BN$, $BK$.

# Thread coarsening: Increasing AI

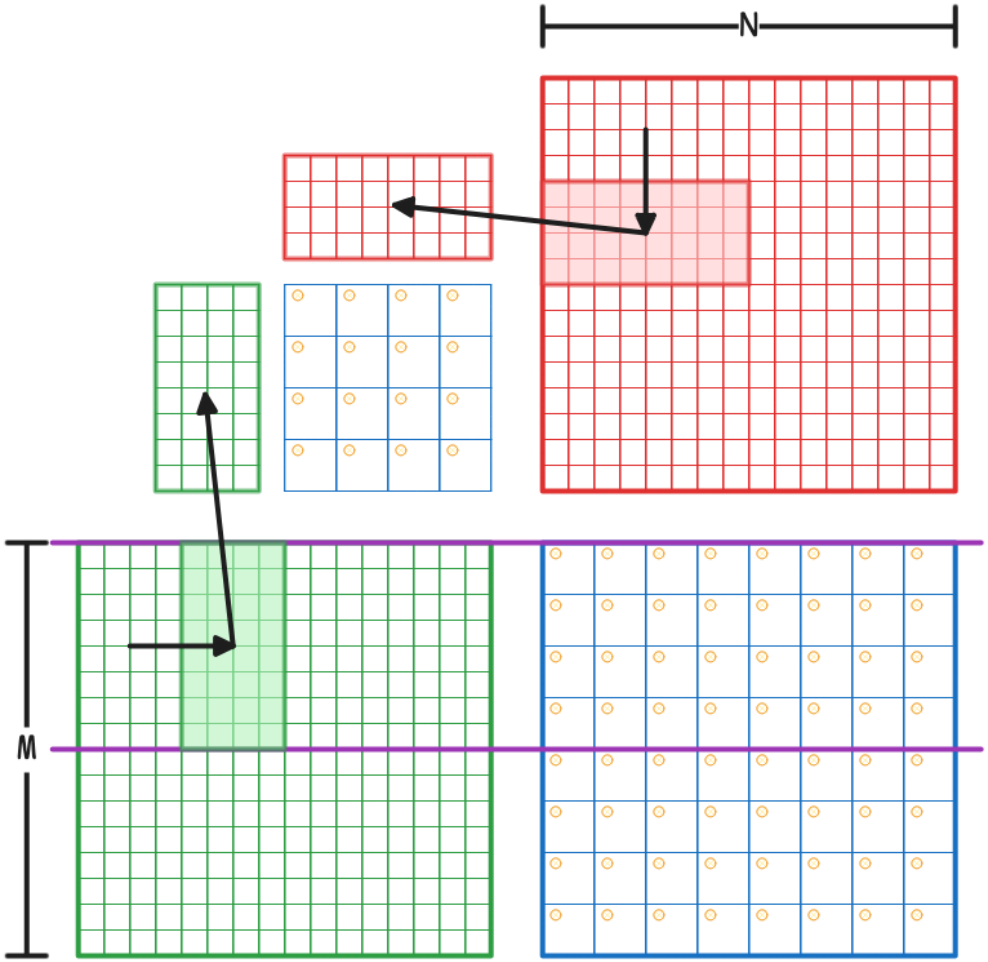
- Basic tiling assigns 1 element of the output to each thread.
- We can *coarsen* each thread, and assign a $TM \times TN$ portion of the output to each thread.
- We load larger chunks into SMEM, increasing data reuse and AI.
- One loaded $A$ value can update several columns; one loaded $B$ value can update several rows.



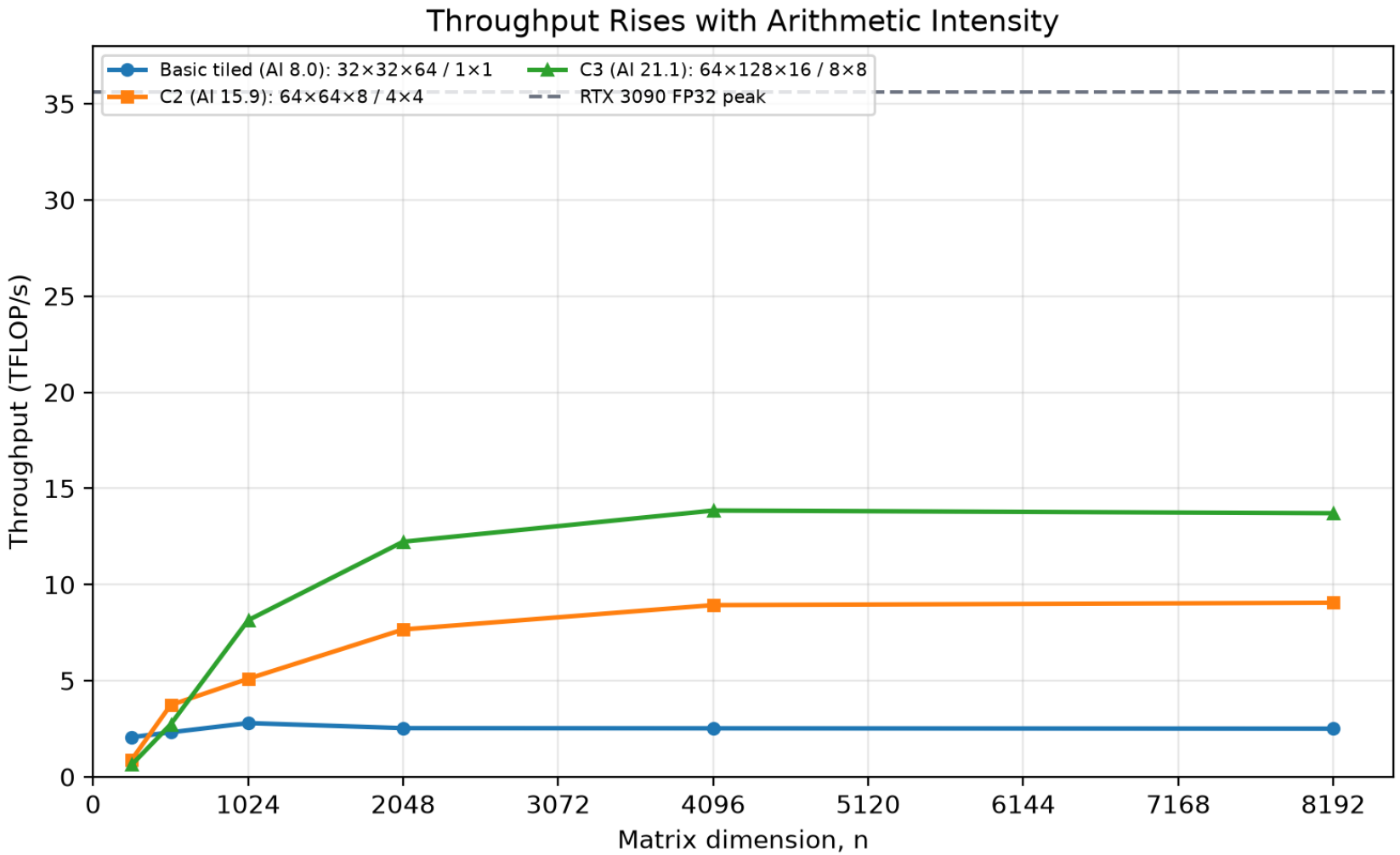
# Coarsened SGEMM visualization



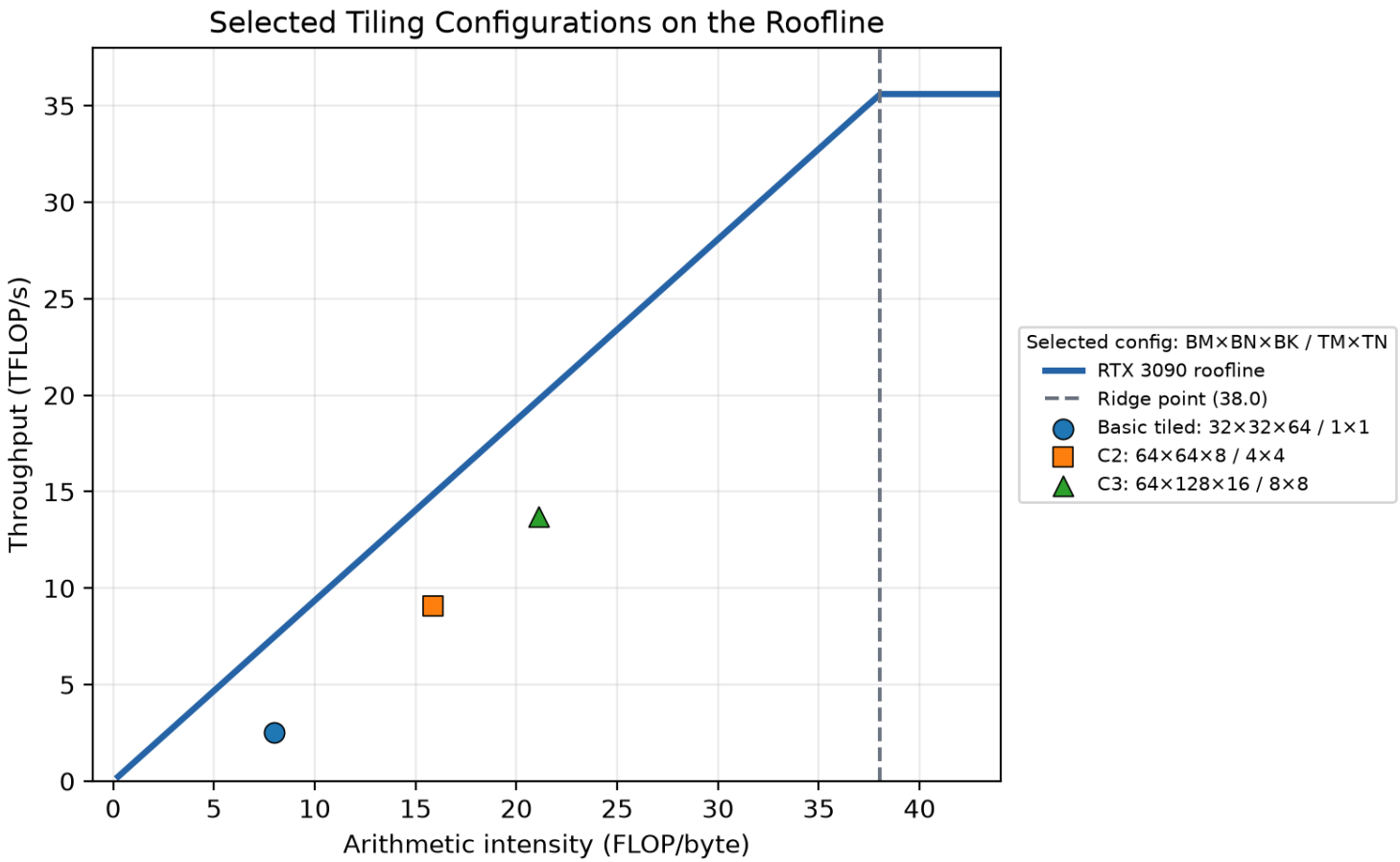
# Coarsened SGEMM visualization



# Higher-AI tiles tend to raise throughput

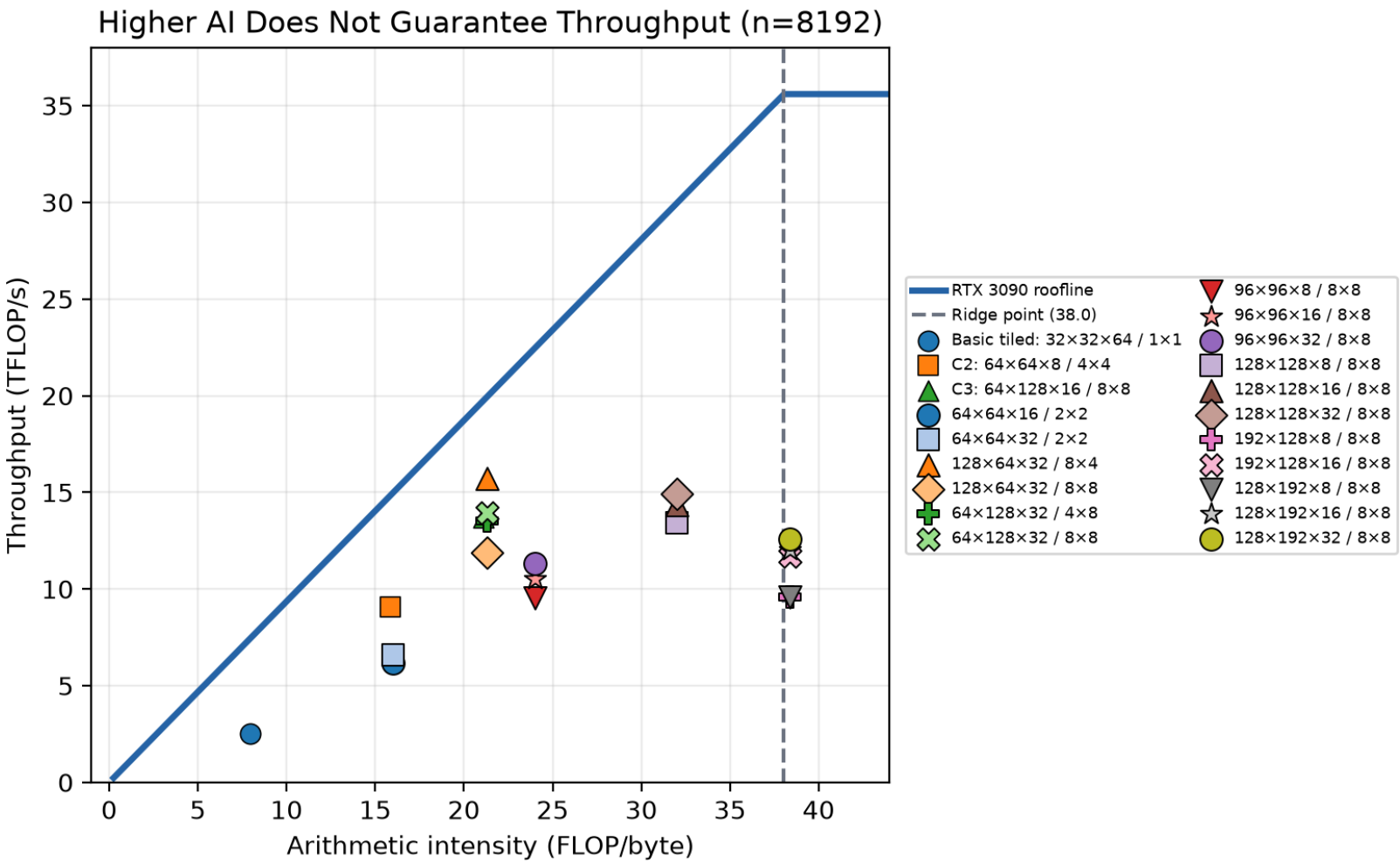


# Tiling configurations on the roofline




Reminder that the Roofline Model only tells us the upper bound!

# Higher AI does not guarantee higher throughput




The roofline is an upper bound, not a performance prediction.

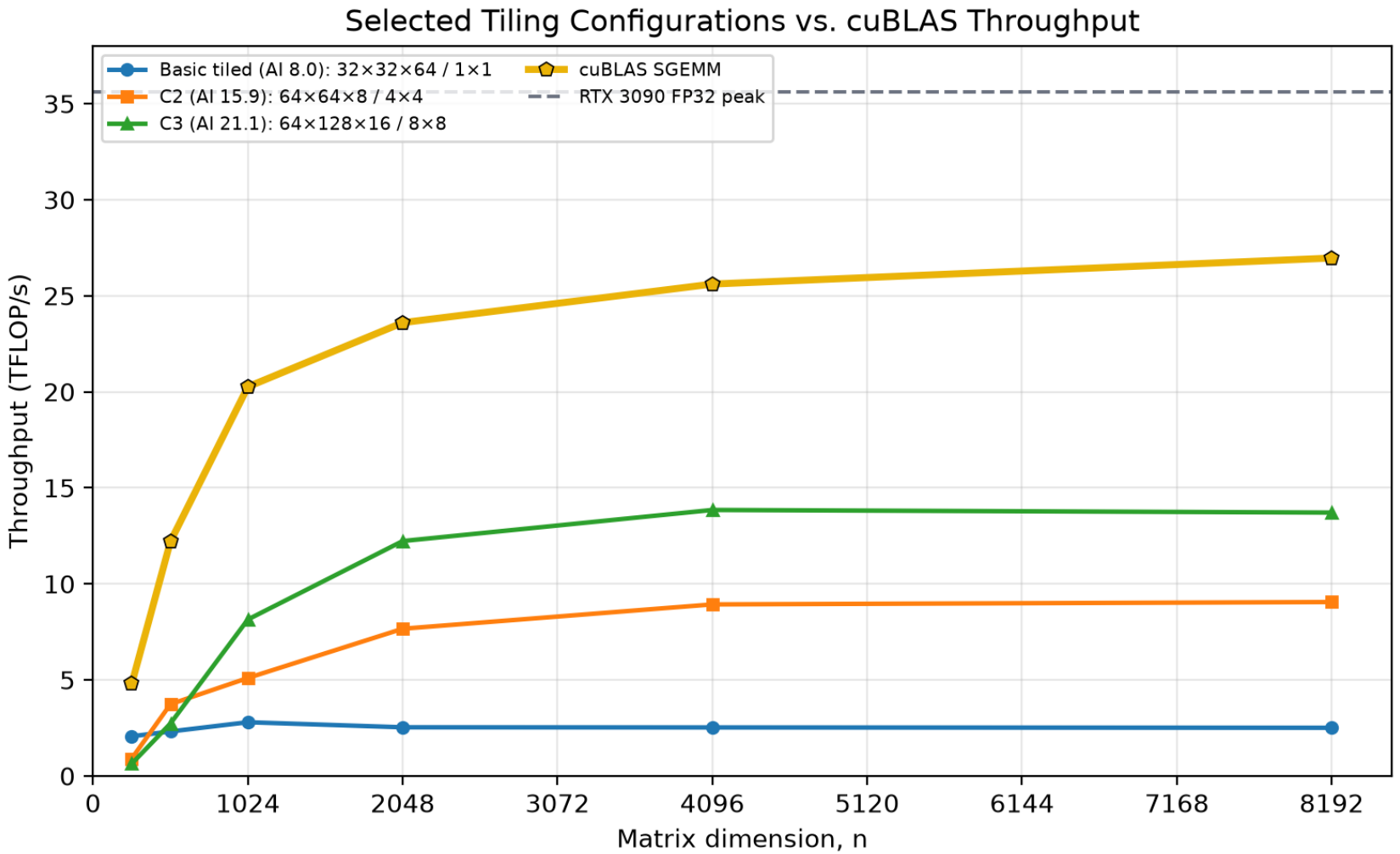
# We have only scratched the surface: cuBLAS

- cuBLAS is NVIDIA's architecture-tuned linear algebra library and the practical performance target for our SGEMM kernels.
- At $n = 8192$, cuBLAS reaches 26.96 TFLOP/s, while the highest-AI selected configuration (C3) reaches 13.71 TFLOP/s, or 50.8% of cuBLAS's throughput.
- cuBLAS completes the multiplication in 40.8 ms versus 80.2 ms for C3, about 1.97× faster.

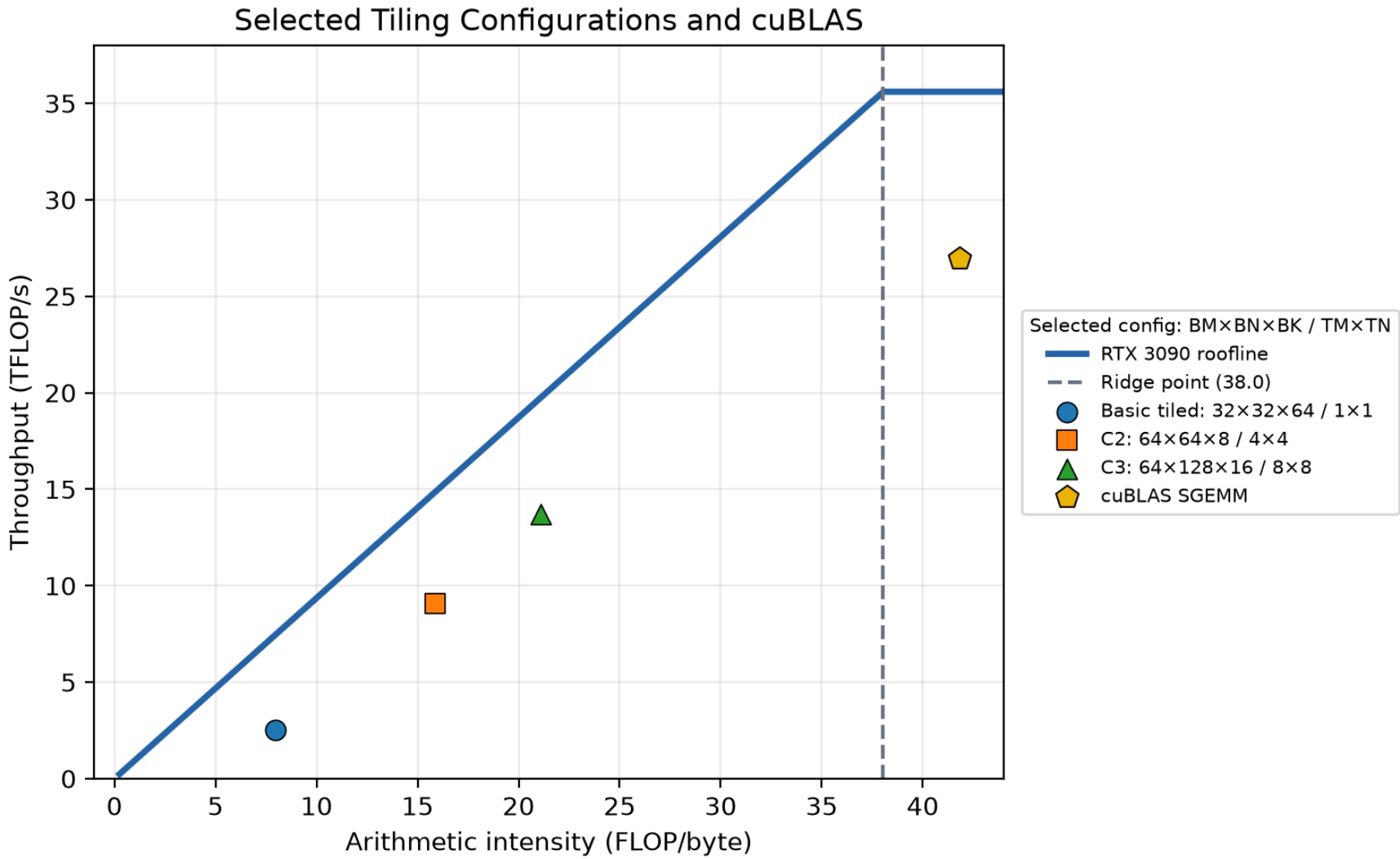
## The remaining gap

Production kernels combine deeper register and warp tiling, vectorized memory access, pipelined loads, and architecture-specific instruction scheduling.

# cuBLAS remains the performance target



# cuBLAS on the roofline




cuBLAS arithmetic intensity is estimated from the profiled $256 \times 128$ output-tile shape.

# We've come a long way

